

Implement EDA_Clustering_Bankruptcy

Sidney Reynaud

2026-06-17

STEP 0: Set up working environment

```
# 0.1 Set a seed for reproducibility
set.seed(123)
```

```
# 0.2 Clear workspace
rm(list = ls())
```

STEP 1: Load required packages

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.3.3
```

```
## Warning: package 'tibble' was built under R version 4.3.3
```

```
## Warning: package 'tidyr' was built under R version 4.3.3
```

```
## Warning: package 'readr' was built under R version 4.3.3
```

```
## Warning: package 'purrr' was built under R version 4.3.3
```

```
## Warning: package 'dplyr' was built under R version 4.3.3
```

```
## Warning: package 'lubridate' was built under R version 4.3.3
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4    ✓ readr      2.1.5
## ✓ forcats   1.0.1    ✓ stringr   1.6.0
## ✓ ggplot2   4.0.3    ✓ tibble    3.2.1
## ✓ lubridate 1.9.4    ✓ tidyr     1.3.1
## ✓ purrr     1.0.4
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(skimr)
library(GGally)
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##   lift
```

```
library(janitor)
```

```
## Warning: package 'janitor' was built under R version 4.3.3
```

```
##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##   chisq.test, fisher.test
```

```
library(yardstick)
```

```
## Warning: package 'yardstick' was built under R version 4.3.3
```

```
##
## Attaching package: 'yardstick'
##
## The following objects are masked from 'package:caret':
##
##   precision, recall, sensitivity, specificity
##
## The following object is masked from 'package:readr':
##
##   spec
```

```
library(factoextra)
```

```
## Warning: package 'factoextra' was built under R version 4.3.3
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

STEP 2: Import data

```
# 2.1 Adjust the path to where american_bankruptcy.csv is saved
bank <- read_csv("american_bankruptcy.csv")
```

```
## Rows: 78682 Columns: 21
## — Column specification —————
## Delimiter: ","
## chr (2): company_name, status_label
## dbl (19): year, company_id, fiscal_year, total_assets, total_liabilities, cu...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# 2.2 Quick peek
glimpse(bank)
```

```
## Rows: 78,682
## Columns: 21
## $ company_name      <chr> "C_1", "C_1", "C_1", "C_1", "C_1", "C_1", ...
## $ status_label      <chr> "alive", "alive", "alive", "alive", "alive...
## $ year              <dbl> 1999, 2000, 2001, 2002, 2003, 2004, 2005, ...
## $ company_id        <dbl> 511.267, 485.856, 436.656, 396.412, 432.20...
## $ fiscal_year       <dbl> 833.107, 713.811, 526.477, 496.747, 523.30...
## $ total_assets      <dbl> 18.373, 18.577, 22.496, 27.172, 26.680, 27...
## $ total_liabilities <dbl> 89.031, 64.367, 27.207, 30.745, 47.491, 61...
## $ current_assets    <dbl> 336.018, 320.590, 286.588, 259.954, 247.24...
## $ current_liabilities <dbl> 35.163, 18.531, -58.939, -12.410, 3.504, 1...
## $ working_capital   <dbl> 128.348, 115.187, 77.528, 66.322, 104.661,...
## $ retained_earnings <dbl> 372.7519, 377.1180, 364.5928, 143.3295, 30...
## $ sales             <dbl> 1024.333, 874.255, 638.721, 606.337, 651.9...
## $ ebit              <dbl> 740.998, 701.854, 710.199, 686.621, 709.29...
## $ net_income        <dbl> 180.447, 179.987, 217.699, 164.658, 248.66...
## $ cash_flow_operations <dbl> 70.658, 45.790, 4.711, 3.573, 20.811, 33.8...
## $ cash_flow_investing <dbl> 191.226, 160.444, 112.244, 109.590, 128.65...
## $ cash_flow_financing <dbl> 163.816, 125.392, 150.464, 203.575, 131.26...
## $ working_capital_to_assets <dbl> 201.026, 204.065, 139.603, 124.106, 131.88...
## $ retained_earnings_to_assets <dbl> 1024.333, 874.255, 638.721, 606.337, 651.9...
## $ ebit_to_assets    <dbl> 401.483, 361.642, 399.964, 391.633, 407.60...
## $ bankrupt_flag     <dbl> 935.302, 809.888, 611.514, 575.592, 604.46...
```

```
head(bank)
```

```
## # A tibble: 6 × 21
##   company_name status_label year company_id fiscal_year total_assets
##   <chr>         <chr>    <dbl>    <dbl>    <dbl>    <dbl>
## 1 C_1          alive    1999      511.      833.      18.4
## 2 C_1          alive    2000      486.      714.      18.6
## 3 C_1          alive    2001      437.      526.      22.5
## 4 C_1          alive    2002      396.      497.      27.2
## 5 C_1          alive    2003      432.      523.      26.7
## 6 C_1          alive    2004      475.      598.      28.0
## # i 15 more variables: total_liabilities <dbl>, current_assets <dbl>,
## #   current_liabilities <dbl>, working_capital <dbl>, retained_earnings <dbl>,
## #   sales <dbl>, ebit <dbl>, net_income <dbl>, cash_flow_operations <dbl>,
## #   cash_flow_investing <dbl>, cash_flow_financing <dbl>,
## #   working_capital_to_assets <dbl>, retained_earnings_to_assets <dbl>,
## #   ebit_to_assets <dbl>, bankrupt_flag <dbl>
```

```
# 3.1 Check column names
colnames(bank)
```

```
## [1] "company_name"      "status_label"
## [3] "year"              "company_id"
## [5] "fiscal_year"       "total_assets"
## [7] "total_liabilities" "current_assets"
## [9] "current_liabilities" "working_capital"
## [11] "retained_earnings" "sales"
## [13] "ebit"              "net_income"
## [15] "cash_flow_operations" "cash_flow_investing"
## [17] "cash_flow_financing" "working_capital_to_assets"
## [19] "retained_earnings_to_assets" "ebit_to_assets"
## [21] "bankrupt_flag"
```

```
# 3.2 Convert status_label to factor (bankrupt vs non-bankrupt)
# Adjust levels if needed based on your actual data
bank <- bank %>%
  mutate(
    status_label = as.factor(status_label),
    year = as.numeric(year)
  )
```

```
# 3.3 Verify structure
str(bank)
```

```
## tibble [78,682 × 21] (S3: tbl_df/tbl/data.frame)
## $ company_name      : chr [1:78682] "C_1" "C_1" "C_1" "C_1" ...
## $ status_label      : Factor w/ 2 levels "alive","failed": 1 1 1 1 1 1 1 1 1 1 ...
## $ year              : num [1:78682] 1999 2000 2001 2002 2003 ...
## $ company_id        : num [1:78682] 511 486 437 396 432 ...
## $ fiscal_year       : num [1:78682] 833 714 526 497 523 ...
## $ total_assets      : num [1:78682] 18.4 18.6 22.5 27.2 26.7 ...
## $ total_liabilities : num [1:78682] 89 64.4 27.2 30.7 47.5 ...
## $ current_assets    : num [1:78682] 336 321 287 260 247 ...
## $ current_liabilities : num [1:78682] 35.2 18.5 -58.9 -12.4 3.5 ...
## $ working_capital   : num [1:78682] 128.3 115.2 77.5 66.3 104.7 ...
## $ retained_earnings : num [1:78682] 373 377 365 143 309 ...
## $ sales             : num [1:78682] 1024 874 639 606 652 ...
## $ ebit              : num [1:78682] 741 702 710 687 709 ...
## $ net_income        : num [1:78682] 180 180 218 165 249 ...
## $ cash_flow_operations : num [1:78682] 70.66 45.79 4.71 3.57 20.81 ...
## $ cash_flow_investing : num [1:78682] 191 160 112 110 129 ...
## $ cash_flow_financing : num [1:78682] 164 125 150 204 131 ...
## $ working_capital_to_assets : num [1:78682] 201 204 140 124 132 ...
## $ retained_earnings_to_assets: num [1:78682] 1024 874 639 606 652 ...
## $ ebit_to_assets    : num [1:78682] 401 362 400 392 408 ...
## $ bankrupt_flag     : num [1:78682] 935 810 612 576 604 ...
```

```
# 4.1 Basic structure and summary
skim(bank)
```

Data summary

Name	bank
Number of rows	78682
Number of columns	21
Column type frequency:	
character	1
factor	1
numeric	19
Group variables	
None	

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
company_name	0	1	3	6	0	8971	0

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
status_label	0	1	FALSE	2	ali: 73462, fai: 5220

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
year	0	1	2007.51	5.74	1999.00	2002.00	2007.00	2012.00	2018	
company_id	0	1	880.36	3928.56	-7.76	18.92	100.45	431.53	169662	
fiscal_year	0	1	1594.53	8930.48	-366.64	17.04	103.66	634.55	374623	
total_assets	0	1	121.23	652.38	0.00	1.19	7.93	47.97	28430	
total_liabilities	0	1	376.76	2012.02	-21913.00	-0.81	15.03	139.66	81730	
current_assets	0	1	201.61	1060.77	0.00	0.00	7.02	74.75	62567	
current_liabilities	0	1	129.38	1265.53	-98696.00	-7.42	1.62	40.14	104821	
working_capital	0	1	286.83	1335.98	-0.01	3.28	22.82	131.58	65812	
retained_earnings	0	1	3414.35	18414.10	0.00	34.98	227.51	1244.89	1073391	
sales	0	1	2364.02	11950.07	-1965.00	27.55	186.60	1046.40	511729	
ebit	0	1	2867.11	12917.94	0.00	37.36	213.20	1171.36	531864	
net_income	0	1	722.48	3242.17	-0.02	0.00	7.59	248.76	166250	
cash_flow_operations	0	1	255.53	1494.64	-25913.00	-2.79	6.52	87.60	71230	
cash_flow_investing	0	1	769.49	3774.70	-21536.00	8.52	63.58	344.07	137106	
cash_flow_financing	0	1	610.07	2938.39	0.00	8.89	43.33	222.82	116866	
working_capital_to_assets	0	1	532.47	6369.16	-102362.00	-68.28	-1.13	146.07	402089	
retained_earnings_to_assets	0	1	2364.02	11950.07	-1965.00	27.55	186.60	1046.40	511729	
ebit_to_assets	0	1	1773.56	8053.68	0.00	13.49	81.99	629.98	337980	
bankrupt_flag	0	1	1987.26	10419.63	-317.20	32.87	168.91	875.52	481580	

```
# 4.2 Frequency table for status_label
table(bank$status_label)
```

```
##
##  alive failed
##  73462   5220
```

```

# -----
# 1. Create data for pie chart
# -----
status_counts <- data.frame(
  status = c("Alive", "Failed"),
  count = c(73462, 5220)
)

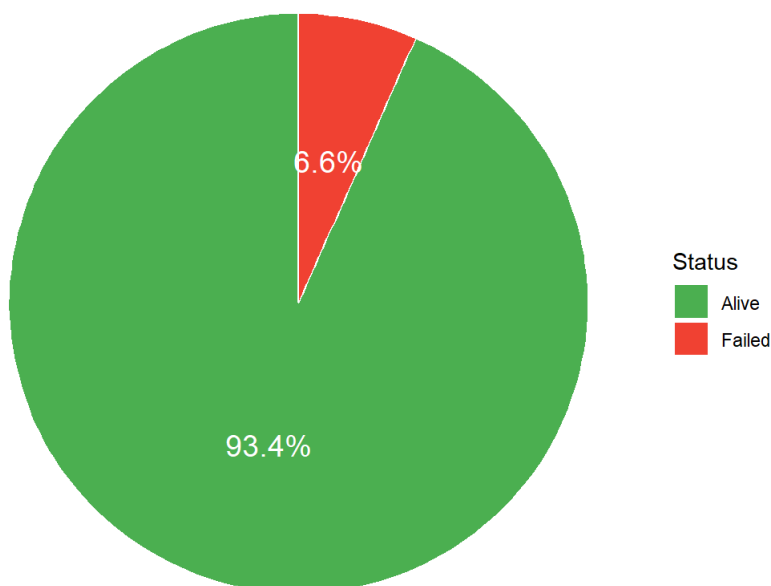
# Calculate percentages
status_counts$percent <- round(status_counts$count / sum(status_counts$count) * 100, 1)

# -----
# 2. Pie chart
# -----
library(ggplot2)

ggplot(status_counts, aes(x = "", y = count, fill = status)) +
  geom_col(width = 1, color = "white") +
  coord_polar(theta = "y") +
  geom_text(aes(label = paste0(percent, "%")),
            position = position_stack(vjust = 0.5),
            color = "white", size = 5) +
  scale_fill_manual(values = c("Alive" = "#4CAF50", "Failed" = "#F44336")) +
  labs(
    title = "Company Status Distribution",
    fill = "Status"
  ) +
  theme_void() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold")
  )

```

Company Status Distribution



```

# 4.3 Count of companies by year
bank %>%
  count(year) %>%
  arrange(year)

```

```
## # A tibble: 20 × 2
##   year      n
##   <dbl> <int>
## 1 1999  5308
## 2 2000  5226
## 3 2001  4897
## 4 2002  4651
## 5 2003  4417
## 6 2004  4348
## 7 2005  4205
## 8 2006  4128
## 9 2007  4009
## 10 2008  3857
## 11 2009  3743
## 12 2010  3625
## 13 2011  3513
## 14 2012  3485
## 15 2013  3484
## 16 2014  3504
## 17 2015  3354
## 18 2016  3191
## 19 2017  3014
## 20 2018  2723
```

STEP 5: Evaluate missing data, outliers, anomalies

5.1 Missing data overview

```
bank %>%
  summarise(across(everything(), ~ sum(is.na(.))))
```

```
## # A tibble: 1 × 21
##   company_name status_label year company_id fiscal_year total_assets
##   <int>         <int> <int>    <int>    <int>    <int>
## 1           0           0  0      0      0      0
## # i 15 more variables: total_liabilities <int>, current_assets <int>,
## #   current_liabilities <int>, working_capital <int>, retained_earnings <int>,
## #   sales <int>, ebit <int>, net_income <int>, cash_flow_operations <int>,
## #   cash_flow_investing <int>, cash_flow_financing <int>,
## #   working_capital_to_assets <int>, retained_earnings_to_assets <int>,
## #   ebit_to_assets <int>, bankrupt_flag <int>
```

5.2 Visualize Missingness pattern (simple)

```
bank %>%
  mutate(across(everything(), ~ ifelse(is.na(.), 1, 0))) %>%
  summarise(across(everything(), mean)) # proportion missing
```

```
## # A tibble: 1 × 21
##   company_name status_label year company_id fiscal_year total_assets
##   <dbl>         <dbl> <dbl>    <dbl>    <dbl>    <dbl>
## 1           0           0  0      0      0      0
## # i 15 more variables: total_liabilities <dbl>, current_assets <dbl>,
## #   current_liabilities <dbl>, working_capital <dbl>, retained_earnings <dbl>,
## #   sales <dbl>, ebit <dbl>, net_income <dbl>, cash_flow_operations <dbl>,
## #   cash_flow_investing <dbl>, cash_flow_financing <dbl>,
## #   working_capital_to_assets <dbl>, retained_earnings_to_assets <dbl>,
## #   ebit_to_assets <dbl>, bankrupt_flag <dbl>
```

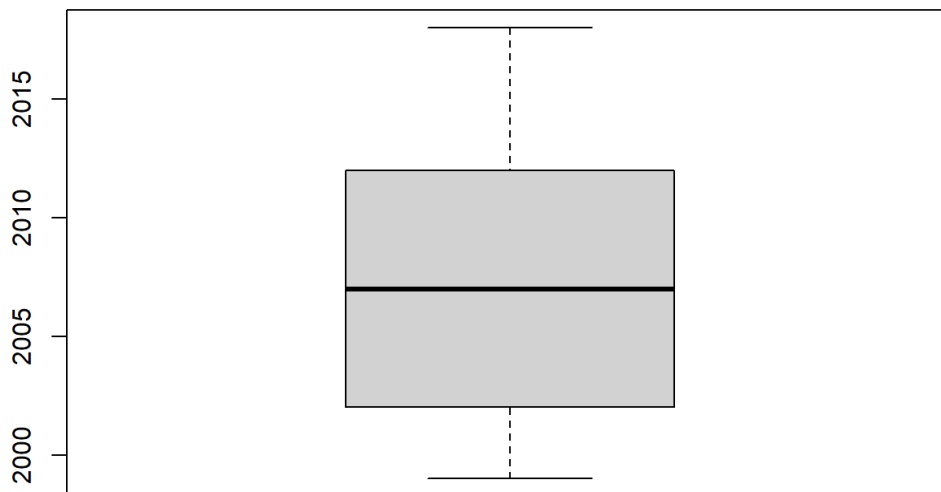
```
# 5.3 Handle missing data
# - Drop rows with missing status_label or year
# - For other numeric predictors (if present), you could impute
```

```
bank_clean <- bank %>%
  filter(!is.na(status_label),
         !is.na(year))
```

```
# If you had more numeric predictors, you could do:
# preProcess_missing <- preProcess(bank_clean, method = "medianImpute")
# bank_clean <- predict(preProcess_missing, bank_clean)
```

```
# 5.4 Outlier detection for numeric variables (here: year)
# Boxplot for year
boxplot(bank_clean$year, main = "Boxplot of Year")
```

Boxplot of Year



```
## 5.5: Run k-means and create cluster column
```

```
# Select numeric predictors for clustering
bank_numeric <- bank_clean %>%
  select(where(is.numeric))
```

```
# Run k-means (choose centers = 2 for alive vs failed)
set.seed(123)
k_res <- kmeans(bank_numeric, centers = 2)
```

```
# Add cluster assignments to the dataset
bank_clean$kmeans_cluster <- as.factor(k_res$cluster)
```

```
# 5.6 Inspect which cluster corresponds to which bankruptcy label
# This step is essential – you must determine whether cluster 1 = alive or failed.
# This tells you the meaning of each cluster.
bank_clean %>%
  group_by(kmeans_cluster) %>%
  summarise(
    alive = sum(status_label == "alive"),
    failed = sum(status_label == "failed")
  )
```

```
## # A tibble: 2 × 3
##   kmeans_cluster alive failed
##   <fct>          <int> <int>
## 1 1              460     0
## 2 2             73002  5220
```

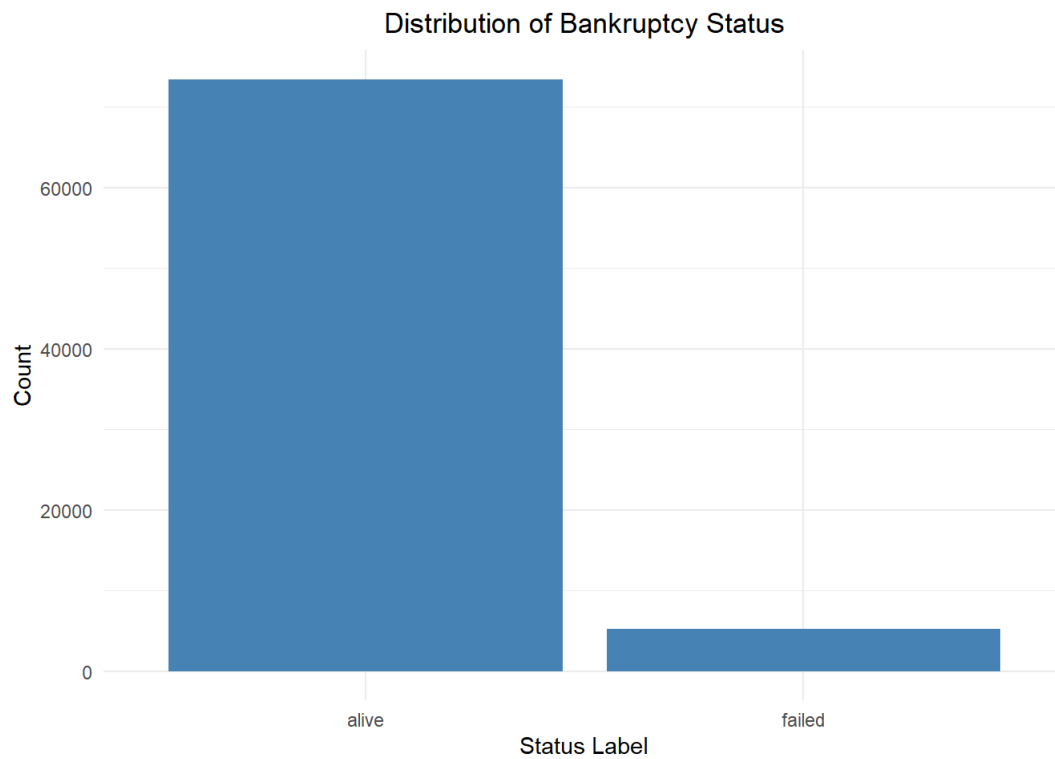
```
# 5.7 Map cluster numbers to actual labels
bank_clean <- bank_clean %>%
  mutate(
    kmeans_label = case_when(
      kmeans_cluster == 1 ~ "alive",
      kmeans_cluster == 2 ~ "failed"
    ) %>% factor(levels = c("alive", "failed"))
  )

# Now your predicted labels match the truth labels.
```

STEP 6: Univariate graphical analysis

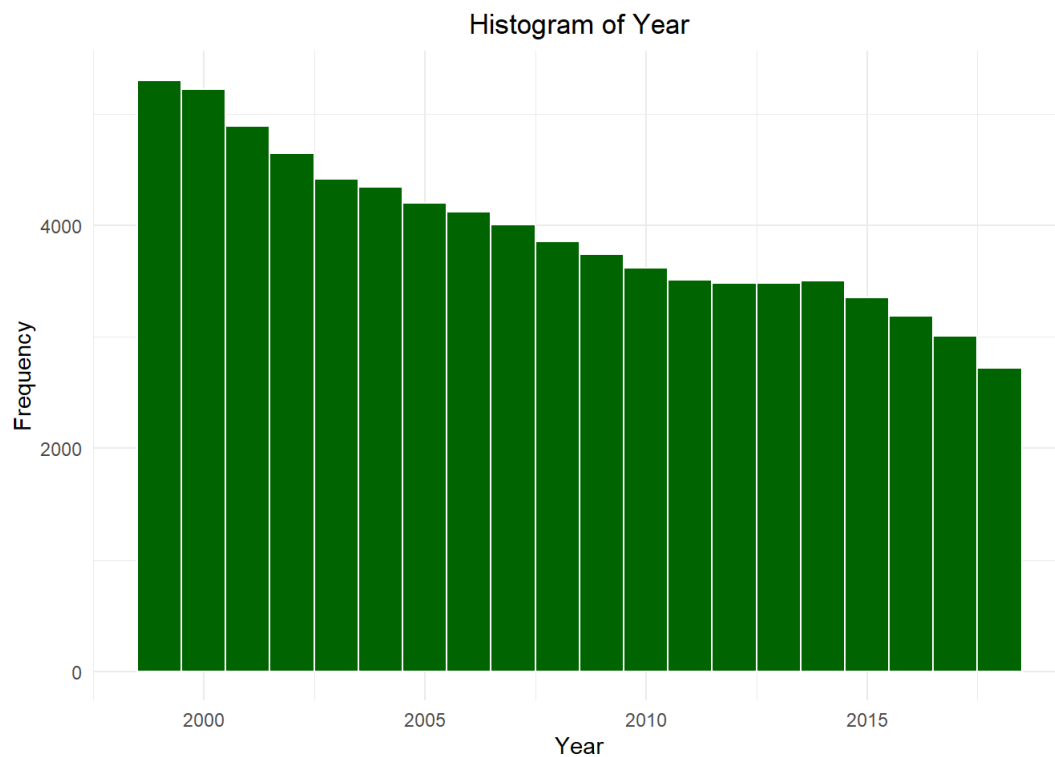
```
# 6.1 Bar plot of status_label

ggplot(bank_clean, aes(x = status_label)) +
  geom_bar(fill = "steelblue") +
  labs(
    title = "Distribution of Bankruptcy Status",
    x = "Status Label",
    y = "Count"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5)
  )
```



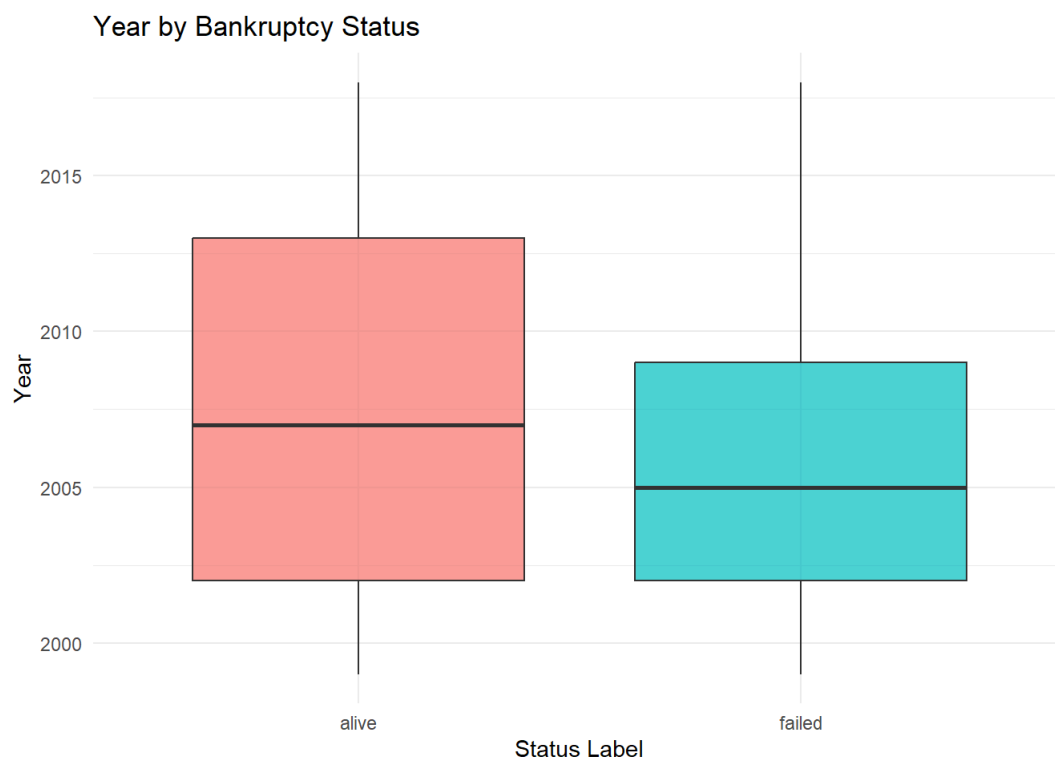
6.2 Histogram of year

```
ggplot(bank_clean, aes(x = year)) +  
  geom_histogram(binwidth = 1, fill = "darkgreen", color = "white") +  
  labs(  
    title = "Histogram of Year",  
    x = "Year",  
    y = "Frequency"  
  ) +  
  theme_minimal() +  
  theme(  
    plot.title = element_text(hjust = 0.5)  
  )
```



STEP 7: Multivariate graphical analysis

```
# 7.1 Year vs status_label (boxplot)
ggplot(bank_clean, aes(x = status_label, y = year, fill = status_label)) +
  geom_boxplot(alpha = 0.7) +
  labs(title = "Year by Bankruptcy Status",
       x = "Status Label",
       y = "Year") +
  theme_minimal() +
  theme(legend.position = "none")
```



7.2 If you have additional numeric predictors, you can use ggpairs:

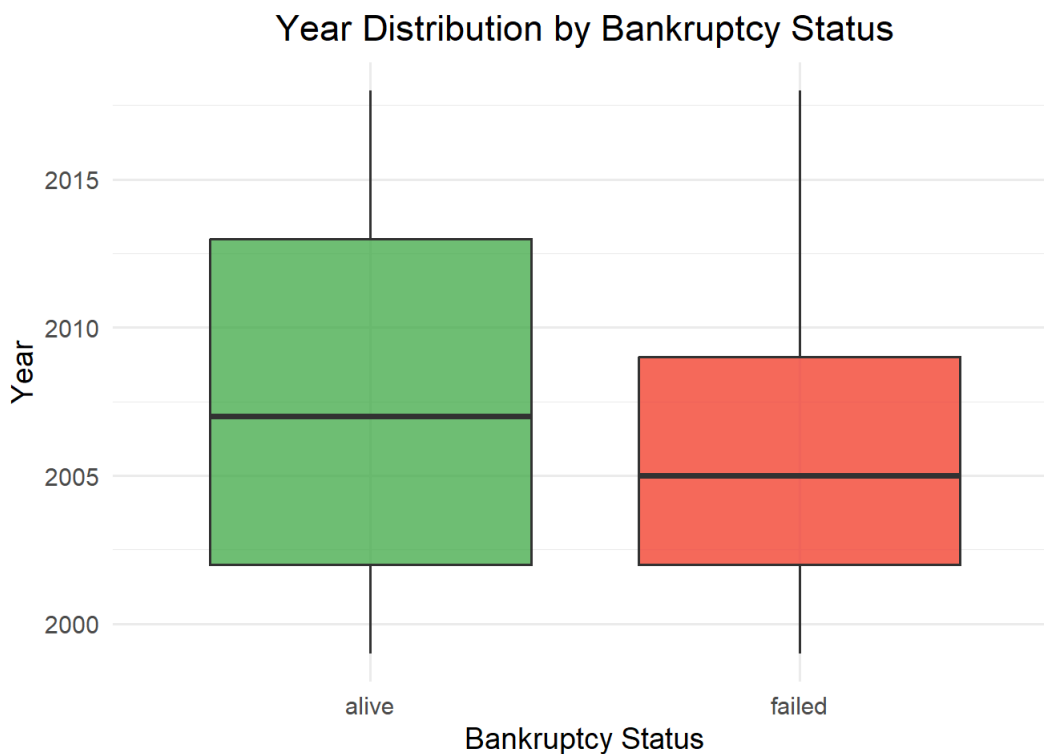
```
bank_clean %>%
```

```
select(status_label, year, other_numeric1, other_numeric2)
%>%
```

```
ggpairs(aes(color = status_label, alpha = 0.5))
```

```
# 7.3 Boxplot: Year by Bankruptcy Status (clean version)

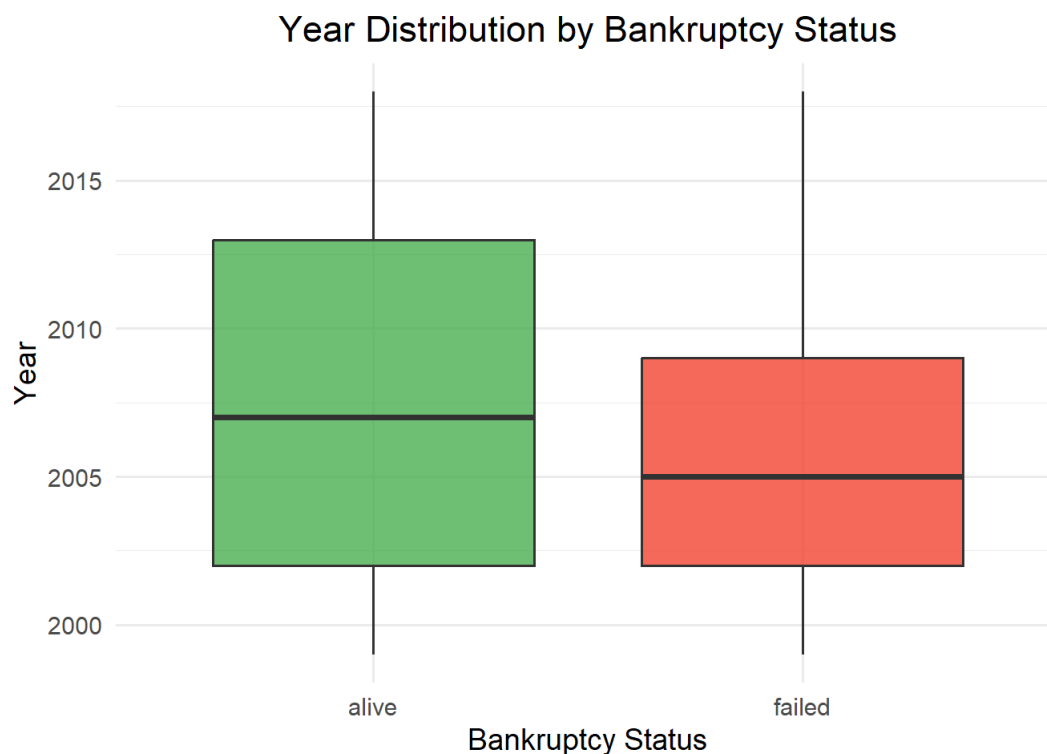
ggplot(bank_clean, aes(x = status_label, y = year, fill = status_label)) +
  geom_boxplot(alpha = 0.8, outlier.color = "black") +
  scale_fill_manual(values = c("alive" = "#4CAF50", "failed" = "#F44336")) +
  labs(
    title = "Year Distribution by Bankruptcy Status",
    x = "Bankruptcy Status",
    y = "Year"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5)
  )
```



7.4 Boxplot Comparing Year by Bankruptcy Status

```
# 7.3 Boxplot: Year by Bankruptcy Status (clean version)
```

```
ggplot(bank_clean, aes(x = status_label, y = year, fill = status_label)) +
  geom_boxplot(alpha = 0.8, outlier.color = "black") +
  scale_fill_manual(values = c("alive" = "#4CAF50", "failed" = "#F44336")) +
  labs(
    title = "Year Distribution by Bankruptcy Status",
    x = "Bankruptcy Status",
    y = "Year"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5)
  )
```



7.4 Correlation Heatmap of Financial Variables

```
# 7.4 Correlation Heatmap of Financial Variables
```

```
library(ggplot2)
library(reshape2)
```

```
## Warning: package 'reshape2' was built under R version 4.3.3
```

```
##
## Attaching package: 'reshape2'
```

```
## The following object is masked from 'package:tidyr':
##
## smiths
```

```

# Select numeric financial variables
financial_vars <- bank_clean %>%
  select(
    total_assets, total_liabilities, current_assets, current_liabilities,
    working_capital, retained_earnings, sales, ebit, net_income,
    cash_flow_operations, cash_flow_investing, cash_flow_financing,
    working_capital_to_assets, retained_earnings_to_assets, ebit_to_assets
  )

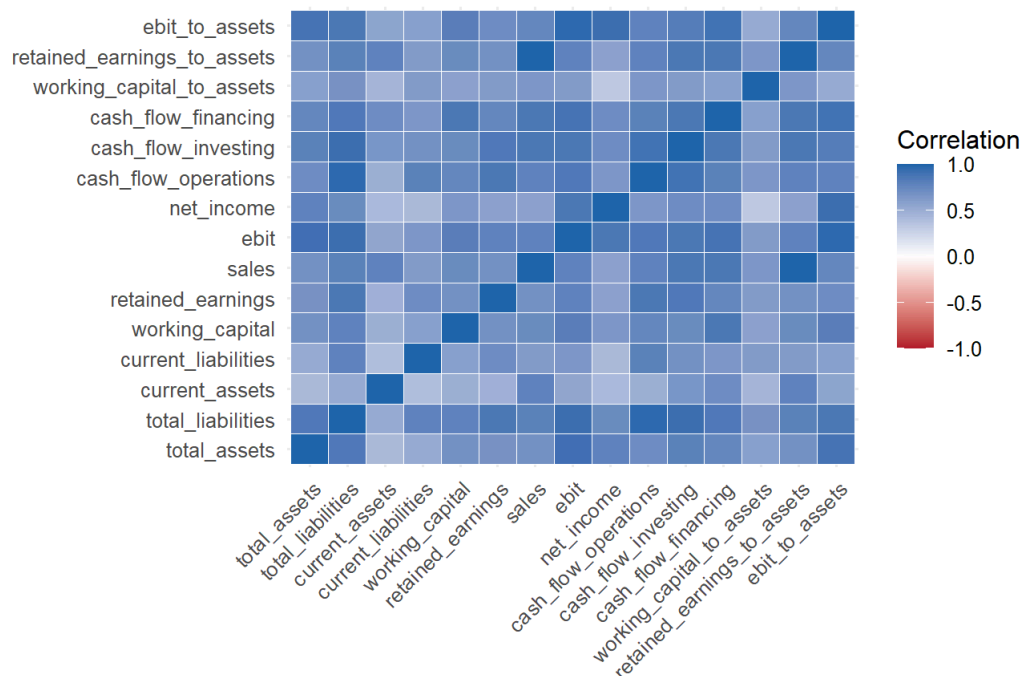
# Compute correlation matrix
corr_matrix <- round(cor(financial_vars, use = "pairwise.complete.obs"), 2)

# Melt for ggplot
corr_melt <- melt(corr_matrix)

# Heatmap
ggplot(corr_melt, aes(Var1, Var2, fill = value)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(
    low = "#B2182B", mid = "white", high = "#2166AC",
    midpoint = 0, limit = c(-1, 1)
  ) +
  labs(
    title = "Correlation Heatmap of Financial Variables",
    x = "",
    y = "",
    fill = "Correlation"
  ) +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

Correlation Heatmap of Financial Variables



STEP 8: Univariate and multivariate statistics

```
# 8.1 Descriptive statistics by status_label
```

```
bank_clean %>%
  group_by(status_label) %>%
  summarise(
    n = n(),
    mean_year = mean(year, na.rm = TRUE),
    sd_year = sd(year, na.rm = TRUE),
    min_year = min(year, na.rm = TRUE),
    max_year = max(year, na.rm = TRUE)
  )
```

```
## # A tibble: 2 × 6
```

```
##   status_label      n mean_year sd_year min_year max_year
##   <fct>         <int>    <dbl>  <dbl>   <dbl>   <dbl>
## 1 alive         73462    2008.   5.78    1999    2018
## 2 failed         5220    2006.   4.81    1999    2018
```

```
# 8.2 Overall descriptive statistics
```

```
summary(bank_clean)
```

```

## company_name      status_label      year      company_id
## Length:78682      alive :73462      Min.   :1999      Min.   : -7.76
## Class :character   failed: 5220      1st Qu.:2002      1st Qu.:  18.92
## Mode  :character                               Median :2007      Median :  100.45
##                                                    Mean  :2008      Mean   :  880.36
##                                                    3rd Qu.:2012      3rd Qu.:  431.53
##                                                    Max.   :2018      Max.   :169662.00
## fiscal_year      total_assets      total_liabilities      current_assets
## Min.   : -366.6      Min.   :  0.000      Min.   : -21913.00      Min.   :  0.00
## 1st Qu.:  17.0      1st Qu.:  1.192      1st Qu.:  -0.81      1st Qu.:  0.00
## Median :  103.7      Median :  7.930      Median :   15.03      Median :  7.02
## Mean   : 1594.5      Mean   : 121.234      Mean   :  376.76      Mean   : 201.61
## 3rd Qu.: 634.5      3rd Qu.: 47.972      3rd Qu.: 139.66      3rd Qu.: 74.75
## Max.   :374623.0      Max.   :28430.000      Max.   : 81730.00      Max.   :62567.00
## current_liabilities      working_capital      retained_earnings      sales
## Min.   : -98696.00      Min.   :  -0.01      Min.   :  0.0      Min.   : -1965.0
## 1st Qu.:  -7.42      1st Qu.:  3.28      1st Qu.:  35.0      1st Qu.:  27.5
## Median :  1.62      Median : 22.82      Median :  227.5      Median : 186.6
## Mean   : 129.38      Mean   : 286.83      Mean   : 3414.4      Mean   : 2364.0
## 3rd Qu.:  40.14      3rd Qu.: 131.58      3rd Qu.: 1244.9      3rd Qu.: 1046.4
## Max.   :104821.00      Max.   :65812.00      Max.   :1073390.5      Max.   :511729.0
## ebit      net_income      cash_flow_operations
## Min.   :  0.0      Min.   : -0.02      Min.   : -25913.00
## 1st Qu.:  37.4      1st Qu.:  0.00      1st Qu.:  -2.79
## Median : 213.2      Median :  7.59      Median :  6.52
## Mean   : 2867.1      Mean   : 722.48      Mean   : 255.53
## 3rd Qu.: 1171.4      3rd Qu.: 248.76      3rd Qu.:  87.60
## Max.   :531864.0      Max.   :166250.00      Max.   : 71230.00
## cash_flow_investing      cash_flow_financing      working_capital_to_assets
## Min.   : -21536.00      Min.   :  0.00      Min.   : -102362.0
## 1st Qu.:  8.52      1st Qu.:  8.89      1st Qu.:  -68.3
## Median :  63.58      Median :  43.33      Median :  -1.1
## Mean   : 769.49      Mean   : 610.07      Mean   :  532.5
## 3rd Qu.: 344.07      3rd Qu.: 222.82      3rd Qu.:  146.1
## Max.   :137106.00      Max.   :116866.00      Max.   : 402089.0
## retained_earnings_to_assets      ebit_to_assets      bankrupt_flag
## Min.   : -1965.0      Min.   :  0.0      Min.   : -317.2
## 1st Qu.:  27.5      1st Qu.: 13.5      1st Qu.:  32.9
## Median : 186.6      Median :  82.0      Median : 168.9
## Mean   : 2364.0      Mean   : 1773.6      Mean   : 1987.3
## 3rd Qu.: 1046.4      3rd Qu.: 630.0      3rd Qu.:  875.5
## Max.   :511729.0      Max.   :337980.0      Max.   :481580.0
## kmeans_cluster      kmeans_label
## 1: 460      alive : 460
## 2:78222      failed:78222
##
##
##
##

```

STEP 9: Feature engineering (at least one new variable)

9.1 Side-by-side Cluster Profile Chart

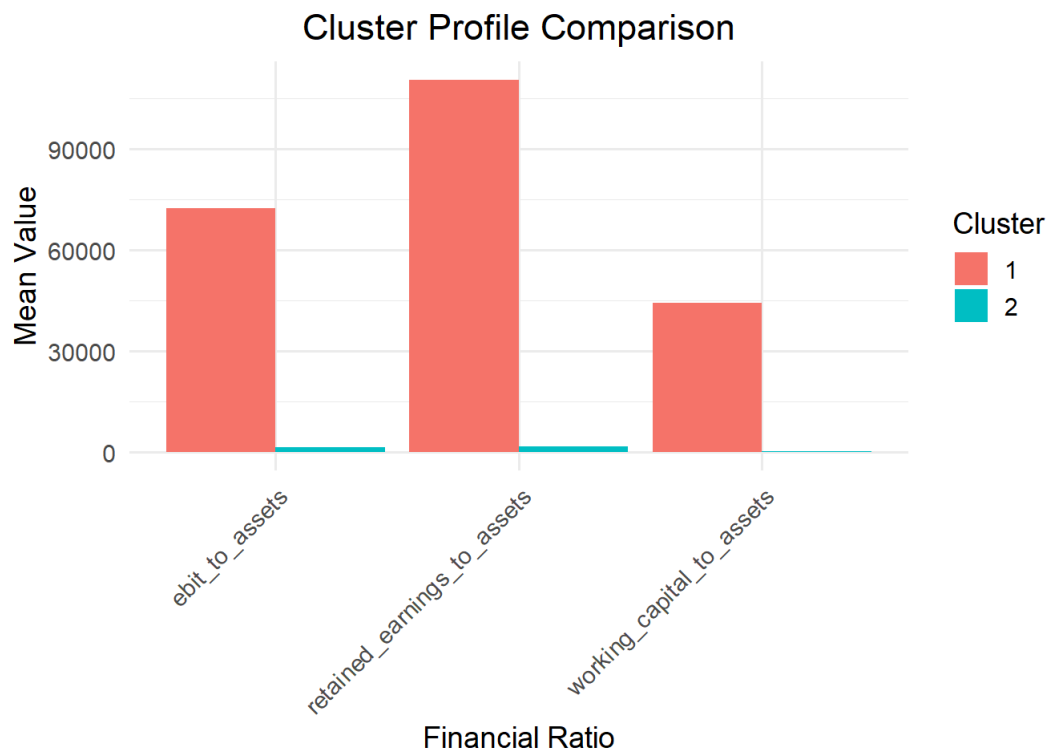
```
library(dplyr)
library(tidyr)
library(ggplot2)

# Select financial variables for profiling
profile_vars <- bank_clean %>%
  select(kmeans_cluster,
         working_capital_to_assets,
         retained_earnings_to_assets,
         ebit_to_assets)

# Compute cluster means
cluster_profile <- profile_vars %>%
  group_by(kmeans_cluster) %>%
  summarise(across(everything(), mean, na.rm = TRUE)) %>%
  pivot_longer(~kmeans_cluster, names_to = "variable", values_to = "value")
```

```
## Warning: There was 1 warning in `summarise()`.
## i In argument: `across(everything(), mean, na.rm = TRUE)` .
## i In group 1: `kmeans_cluster = 1` .
## Caused by warning:
## ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
## Supply arguments directly to `.fns` through an anonymous function instead.
##
## # Previously
## across(a:b, mean, na.rm = TRUE)
##
## # Now
## across(a:b, \(x) mean(x, na.rm = TRUE))
```

```
# Plot side-by-side bar chart
ggplot(cluster_profile, aes(x = variable, y = value, fill = kmeans_cluster)) +
  geom_col(position = "dodge") +
  labs(
    title = "Cluster Profile Comparison",
    x = "Financial Ratio",
    y = "Mean Value",
    fill = "Cluster"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(hjust = 0.5)
  )
```



9.2 Radar Chart Comparing Financial Ratios Across Clusters

```
# 9.2 Radar Chart for Cluster Profiles
```

```
library(fmsb)
```

```
## Warning: package 'fmsb' was built under R version 4.3.3
```

```
## Registered S3 methods overwritten by 'fmsb':
##   method      from
##   print.roc   pROC
##   plot.roc    pROC
```

```
library(dplyr)

# Compute cluster means
radar_data <- bank_clean %>%
  group_by(kmeans_cluster) %>%
  summarise(
    working_capital_to_assets = mean(working_capital_to_assets, na.rm = TRUE),
    retained_earnings_to_assets = mean(retained_earnings_to_assets, na.rm = TRUE),
    ebit_to_assets = mean(ebit_to_assets, na.rm = TRUE)
  )

# Prepare radar chart format
max_vals <- apply(radar_data[, -1], 2, max)
min_vals <- apply(radar_data[, -1], 2, min)

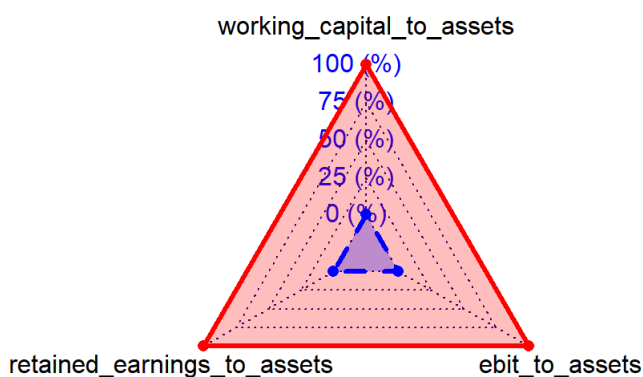
radar_plot <- rbind(max_vals, min_vals, radar_data[, -1])

# Correct row names
rownames(radar_plot) <- c(
  "max",
  "min",
  paste0("Cluster_", radar_data$kmeans_cluster)
)
```

```
## Warning: Setting row names on a tibble is deprecated.
```

```
# Plot radar chart
radarchart(
  radar_plot,
  axistype = 1,
  pcol = c("red", "blue", "darkgreen"),
  pfcol = c("#FF000040", "#0000FF40", "#00800040"),
  plwd = 3,
  title = "Radar Chart: Financial Ratio Profiles by Cluster"
)
```

Radar Chart: Financial Ratio Profiles by Cluster



```
# 9.3 Years_from_2000 = year - 2000
bank_fe <- bank_clean %>%
  mutate(
    years_from_2000 = year - 2000
  )
```

```
# 9.4 Check new variable
summary(bank_fe$years_from_2000)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -1.000   2.000    7.000   7.506  12.000  18.000
```

STEP 10: Prepare data for modeling (remove non-predictors)

```
# 10.1 Remove company_name (ID-like) from predictors
# Keep status_label as outcome; year and engineered features as predictors
# If you have more predictors, include them here.
```

```
model_data <- bank_fe %>%
  select(-company_name)
```

```
# 10.2 Confirm structure
str(model_data)
```

```
## tibble [78,682 × 23] (S3: tbl_df/tbl/data.frame)
## $ status_label      : Factor w/ 2 levels "alive","failed": 1 1 1 1 1 1 1 1 1 1 ...
## $ year              : num [1:78682] 1999 2000 2001 2002 2003 ...
## $ company_id        : num [1:78682] 511 486 437 396 432 ...
## $ fiscal_year        : num [1:78682] 833 714 526 497 523 ...
## $ total_assets       : num [1:78682] 18.4 18.6 22.5 27.2 26.7 ...
## $ total_liabilities  : num [1:78682] 89 64.4 27.2 30.7 47.5 ...
## $ current_assets     : num [1:78682] 336 321 287 260 247 ...
## $ current_liabilities : num [1:78682] 35.2 18.5 -58.9 -12.4 3.5 ...
## $ working_capital    : num [1:78682] 128.3 115.2 77.5 66.3 104.7 ...
## $ retained_earnings  : num [1:78682] 373 377 365 143 309 ...
## $ sales              : num [1:78682] 1024 874 639 606 652 ...
## $ ebit               : num [1:78682] 741 702 710 687 709 ...
## $ net_income         : num [1:78682] 180 180 218 165 249 ...
## $ cash_flow_operations : num [1:78682] 70.66 45.79 4.71 3.57 20.81 ...
## $ cash_flow_investing : num [1:78682] 191 160 112 110 129 ...
## $ cash_flow_financing : num [1:78682] 164 125 150 204 131 ...
## $ working_capital_to_assets : num [1:78682] 201 204 140 124 132 ...
## $ retained_earnings_to_assets: num [1:78682] 1024 874 639 606 652 ...
## $ ebit_to_assets     : num [1:78682] 401 362 400 392 408 ...
## $ bankrupt_flag      : num [1:78682] 935 810 612 576 604 ...
## $ kmeans_cluster     : Factor w/ 2 levels "1","2": 2 2 2 2 2 2 2 2 2 2 ...
## $ kmeans_label       : Factor w/ 2 levels "alive","failed": 2 2 2 2 2 2 2 2 2 2 ...
## $ years_from_2000    : num [1:78682] -1 0 1 2 3 4 5 6 7 8 ...
```

STEP 11: Split data into training and test sets (no leakage)

```
# 11.1
set.seed(123)

train_index <- createDataPartition(model_data$status_label, p = 0.7, list = FALSE)

train_data <- model_data[train_index, ]
test_data <- model_data[-train_index, ]
```

```
# 11.2 Check class balance
prop.table(table(train_data$status_label))
```

```
##
##      alive      failed
## 0.93365772 0.06634228
```

```
prop.table(table(test_data$status_label))
```

```
##
##      alive      failed
## 0.93365531 0.06634469
```

STEP 12: Transformation / scaling for clustering

```
# K-means requires numeric predictors only.
# We'll create a numeric-only matrix for clustering (excluding status_label).
```

```
# 12.1 Identify numeric predictors
numeric_vars <- train_data %>%
  select(where(is.numeric)) %>%
  colnames()

numeric_vars
```

```
## [1] "year"                "company_id"
## [3] "fiscal_year"         "total_assets"
## [5] "total_liabilities"   "current_assets"
## [7] "current_liabilities" "working_capital"
## [9] "retained_earnings"   "sales"
## [11] "ebit"                "net_income"
## [13] "cash_flow_operations" "cash_flow_investing"
## [15] "cash_flow_financing" "working_capital_to_assets"
## [17] "retained_earnings_to_assets" "ebit_to_assets"
## [19] "bankrupt_flag"       "years_from_2000"
```

```
# 12.2 Scale numeric predictors on training data
preproc_scale <- preProcess(train_data[, numeric_vars], method = c("center", "scale"))

train_scaled <- predict(preproc_scale, train_data[, numeric_vars])
test_scaled <- predict(preproc_scale, test_data[, numeric_vars])
```

```
# 12.3 Check scaled summary
summary(train_scaled)
```

```
##      year      company_id      fiscal_year      total_assets
## Min.   :-1.47523  Min.    :-0.2261  Min.    :-0.2169  Min.    :-0.1860
## 1st Qu.: -0.95385  1st Qu.: -0.2193  1st Qu.: -0.1745  1st Qu.: -0.1842
## Median : -0.08488  Median : -0.1987  Median : -0.1650  Median : -0.1738
## Mean   :  0.00000  Mean   :  0.0000  Mean   :  0.0000  Mean   :  0.0000
## 3rd Qu.:  0.78409  3rd Qu.: -0.1145  3rd Qu.: -0.1062  3rd Qu.: -0.1124
## Max.    :  1.82685  Max.    :43.1339  Max.    :41.2458  Max.    :39.5607
## total_liabilities current_assets current_liabilities working_capital
## Min.   :-10.8344  Min.    :-0.1905  Min.    :-72.92781  Min.    :-0.2125
## 1st Qu.: -0.1840  1st Qu.: -0.1905  1st Qu.: -0.10156  1st Qu.: -0.2100
## Median : -0.1764  Median : -0.1840  Median : -0.09494  Median : -0.1958
## Mean   :  0.0000  Mean   :  0.0000  Mean   :  0.00000  Mean   :  0.0000
## 3rd Qu.: -0.1162  3rd Qu.: -0.1210  3rd Qu.: -0.06679  3rd Qu.: -0.1157
## Max.    : 39.5411  Max.    :44.1507  Max.    : 77.25542  Max.    :48.6961
## retained_earnings sales      ebit      net_income
## Min.   :-0.1833  Min.    :-0.1961  Min.    :-0.2216  Min.    :-0.2218
## 1st Qu.: -0.1814  1st Qu.: -0.1936  1st Qu.: -0.2187  1st Qu.: -0.2218
## Median : -0.1713  Median : -0.1807  Median : -0.2051  Median : -0.2194
## Mean   :  0.0000  Mean   :  0.0000  Mean   :  0.0000  Mean   :  0.0000
## 3rd Qu.: -0.1178  3rd Qu.: -0.1095  3rd Qu.: -0.1315  3rd Qu.: -0.1452
## Max.    :56.8311  Max.    :42.1463  Max.    :34.2266  Max.    :38.5344
## cash_flow_operations cash_flow_investing cash_flow_financing
## Min.   :-16.9434  Min.    :-5.8494  Min.    :-0.2074
## 1st Qu.: -0.1681  1st Qu.: -0.2004  1st Qu.: -0.2044
## Median : -0.1621  Median : -0.1862  Median : -0.1928
## Mean   :  0.0000  Mean   :  0.0000  Mean   :  0.0000
## 3rd Qu.: -0.1101  3rd Qu.: -0.1128  3rd Qu.: -0.1324
## Max.    : 45.9510  Max.    :35.7469  Max.    :39.6065
## working_capital_to_assets retained_earnings_to_assets ebit_to_assets
## Min.   :-15.77512  Min.    :-0.1961  Min.    :-0.2189
## 1st Qu.: -0.09470  1st Qu.: -0.1936  1st Qu.: -0.2172
## Median : -0.08452  Median : -0.1807  Median : -0.2088
## Mean   :  0.00000  Mean   :  0.0000  Mean   :  0.0000
## 3rd Qu.: -0.06214  3rd Qu.: -0.1095  3rd Qu.: -0.1420
## Max.    : 60.96658  Max.    :42.1463  Max.    :37.1684
## bankrupt_flag      years_from_2000
## Min.   :-0.2193  Min.    :-1.47523
## 1st Qu.: -0.1861  1st Qu.: -0.95385
## Median : -0.1733  Median : -0.08488
## Mean   :  0.0000  Mean   :  0.00000
## 3rd Qu.: -0.1063  3rd Qu.:  0.78409
## Max.    :45.5866  Max.    : 1.82685
```

STEP 13: Build k-means clustering model (k = 2)

```
# 13.1
set.seed(123)

k2_model <- kmeans(train_scaled, centers = 2, nstart = 25)

# Inspect model
k2_model$size
```

```
## [1] 54817 261
```

```
k2_model$centers
```

```
##          year company_id fiscal_year total_assets total_liabilities
## 1 -0.003406733 -0.0498006 -0.04132953 -0.04553258 -0.05060885
## 2  0.715505277 10.4594608  8.68030992  9.56306225 10.62921604
## current_assets current_liabilities working_capital retained_earnings
## 1 -0.03612943 -0.03949464 -0.04598965 -0.04541464
## 2  7.58814944  8.29493337  9.65906007  9.53829209
## sales ebit net_income cash_flow_operations cash_flow_investing
## 1 -0.0473636 -0.05033195 -0.03738306 -0.04824326 -0.05208108
## 2  9.9476259 10.57105843  7.85144578 10.13237944 10.93842300
## cash_flow_financing working_capital_to_assets retained_earnings_to_assets
## 1 -0.05160921 -0.03892747 -0.0473636
## 2 10.83931761 8.17581213 9.9476259
## ebit_to_assets bankrupt_flag years_from_2000
## 1 -0.04821516 -0.04451273 -0.003406733
## 2 10.12647676 9.34886674 0.715505277
```

STEP 14: Attach cluster assignments to training data

```
# 14.1
train_data$cluster <- as.factor(k2_model$cluster)
```

```
# 14.2 Compare clusters with true status_label
table(train_data$status_label, train_data$cluster)
```

```
##
##          1      2
## alive 51163 261
## failed 3654   0
```

STEP 15: Interpret clusters (high-level)

```
# 15.1 Cluster-wise means of numeric variables
train_data %>%
  mutate(across(all_of(numeric_vars), as.numeric)) %>%
  group_by(cluster) %>%
  summarise(across(all_of(numeric_vars), mean, na.rm = TRUE))
```

```
## # A tibble: 2 × 21
##   cluster year company_id fiscal_year total_assets total_liabilities
##   <fct>   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 1      2007.      682.      1221.      91.3      274.
## 2 2      2012.     41806.     80100.     6340.     22246.
## # i 15 more variables: current_assets <dbl>, current_liabilities <dbl>,
## #   working_capital <dbl>, retained_earnings <dbl>, sales <dbl>, ebit <dbl>,
## #   net_income <dbl>, cash_flow_operations <dbl>, cash_flow_investing <dbl>,
## #   cash_flow_financing <dbl>, working_capital_to_assets <dbl>,
## #   retained_earnings_to_assets <dbl>, ebit_to_assets <dbl>,
## #   bankrupt_flag <dbl>, years_from_2000 <dbl>
```

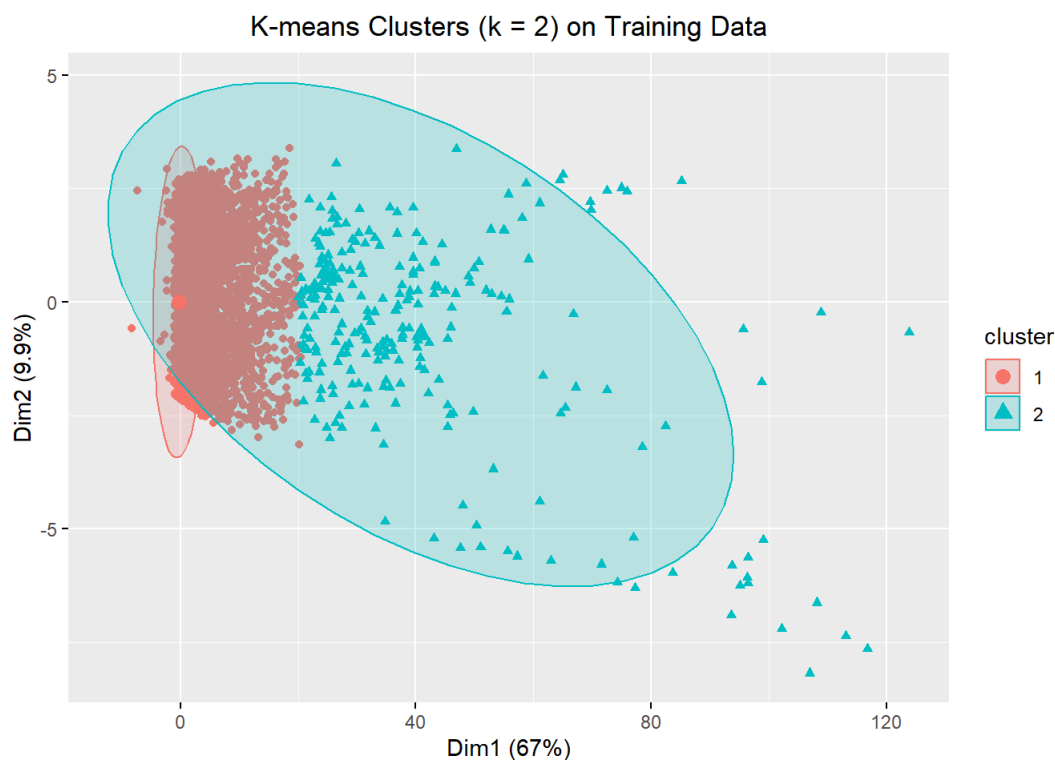
```
# 15.2 Cluster composition by status_label
train_data %>%
  count(cluster, status_label) %>%
  group_by(cluster) %>%
  mutate(prop = n / sum(n))
```

```
## # A tibble: 3 × 4
## # Groups:   cluster [2]
##   cluster status_label     n  prop
##   <fct>   <fct>       <int> <dbl>
## 1 1      alive       51163 0.933
## 2 1      failed       3654 0.0667
## 3 2      alive         261 1
```

STEP 16: Visualize final clusters (using first two numeric features)

16.1 If you have more than two numeric predictors, you can use PCA.
Here, we'll just use factoextra for a basic cluster plot.

```
fviz_cluster(
  list(data = train_scaled, cluster = k2_model$cluster),
  geom = "point",
  ellipse.type = "norm",
  main = "K-means Clusters (k = 2) on Training Data",
  ggtheme = theme(plot.title = element_text(hjust = 0.5))
)
```



STEP 17: Use clusters to predict status on test set

```
# 17.1 Define train_df and test_df using the objects in your workspace
train_df <- train_data
test_df  <- test_data
```

```
# 17.2 Scale training and test data using the same scaler

# Keep only numeric columns
train_df_num <- train_df[sapply(train_df, is.numeric)]
test_df_num <- test_df[sapply(test_df, is.numeric)]

# Scale training data
train_scaled <- scale(train_df_num)

# Save scaling parameters
train_center <- attr(train_scaled, "scaled:center")
train_scale <- attr(train_scaled, "scaled:scale")

# Apply same scaling to test data
test_scaled <- scale(test_df_num, center = train_center, scale = train_scale)
```

```
# 17.3 Fit k-means on the training data
set.seed(123)
k2_model <- kmeans(train_scaled, centers = 2, nstart = 25)
```

```
# 17.4 Define a safe, efficient cluster-assignment function
assign_cluster <- function(newdata, centers) {
  apply(newdata, 1, function(row) {
    dists <- colSums((t(centers) - row)^2)
    which.min(dists)
  })
}
```

```
# 17.5 Assign clusters to the test data
test_clusters <- assign_cluster(test_scaled, k2_model$centers)
```

```
# 17.6 Attach cluster labels back to the test set
test_df$cluster <- test_clusters
```

```
# 17.7 Evaluate cluster separation (Silhouette score:)
library(cluster)

# Sample 2000 points (or fewer if needed)
set.seed(123)
idx <- sample(nrow(train_scaled), 2000)

sil <- silhouette(k2_model$cluster[idx], dist(train_scaled[idx, ]))
mean(sil[, 3])
```

```
## [1] 0.9384821
```

```
WCSS <- sapply(1:5, function(k) kmeans(train_scaled, centers = k)$tot.withinss)
```

STEP 18: Model evaluation on test set

```
# 18.1 How to diagnose and Look for the actual cluster column.
names(bank_clean)
```

```
## [1] "company_name"      "status_label"
## [3] "year"               "company_id"
## [5] "fiscal_year"        "total_assets"
## [7] "total_liabilities"  "current_assets"
## [9] "current_liabilities" "working_capital"
## [11] "retained_earnings"  "sales"
## [13] "ebit"               "net_income"
## [15] "cash_flow_operations" "cash_flow_investing"
## [17] "cash_flow_financing" "working_capital_to_assets"
## [19] "retained_earnings_to_assets" "ebit_to_assets"
## [21] "bankrupt_flag"      "kmeans_cluster"
## [23] "kmeans_label"
```

18.2 Inspect and should see the cluster column

```
str(bank_clean)
```

```
## tibble [78,682 × 23] (S3: tbl_df/tbl/data.frame)
## $ company_name      : chr [1:78682] "C_1" "C_1" "C_1" "C_1" ...
## $ status_label      : Factor w/ 2 levels "alive","failed": 1 1 1 1 1 1 1 1 1 1 ...
## $ year              : num [1:78682] 1999 2000 2001 2002 2003 ...
## $ company_id        : num [1:78682] 511 486 437 396 432 ...
## $ fiscal_year       : num [1:78682] 833 714 526 497 523 ...
## $ total_assets      : num [1:78682] 18.4 18.6 22.5 27.2 26.7 ...
## $ total_liabilities : num [1:78682] 89 64.4 27.2 30.7 47.5 ...
## $ current_assets    : num [1:78682] 336 321 287 260 247 ...
## $ current_liabilities : num [1:78682] 35.2 18.5 -58.9 -12.4 3.5 ...
## $ working_capital   : num [1:78682] 128.3 115.2 77.5 66.3 104.7 ...
## $ retained_earnings : num [1:78682] 373 377 365 143 309 ...
## $ sales             : num [1:78682] 1024 874 639 606 652 ...
## $ ebit              : num [1:78682] 741 702 710 687 709 ...
## $ net_income        : num [1:78682] 180 180 218 165 249 ...
## $ cash_flow_operations : num [1:78682] 70.66 45.79 4.71 3.57 20.81 ...
## $ cash_flow_investing : num [1:78682] 191 160 112 110 129 ...
## $ cash_flow_financing : num [1:78682] 164 125 150 204 131 ...
## $ working_capital_to_assets : num [1:78682] 201 204 140 124 132 ...
## $ retained_earnings_to_assets : num [1:78682] 1024 874 639 606 652 ...
## $ ebit_to_assets     : num [1:78682] 401 362 400 392 408 ...
## $ bankrupt_flag      : num [1:78682] 935 810 612 576 604 ...
## $ kmeans_cluster     : Factor w/ 2 levels "1","2": 2 2 2 2 2 2 2 2 2 2 ...
## $ kmeans_label       : Factor w/ 2 levels "alive","failed": 2 2 2 2 2 2 2 2 2 2 ...
```

18.3 Inspect cluster meaning

This tells you: Cluster 1 is mostly alive or failed and Cluster 2 is mostly alive or failed

```
bank_clean %>%
  group_by(kmeans_cluster) %>%
  summarise(
    alive = sum(status_label == "alive"),
    failed = sum(status_label == "failed")
  )
```

```
## # A tibble: 2 × 3
##   kmeans_cluster alive failed
##   <fct>          <int> <int>
## 1 1              460     0
## 2 2             73002  5220
```

```
# 18.4 Map clusters to labels
bank_clean <- bank_clean %>%
  mutate(
    kmeans_label = case_when(
      kmeans_cluster == 1 ~ "alive",
      kmeans_cluster == 2 ~ "failed"
    ) %>% factor(levels = c("alive", "failed"))
  )
```

```
# 18.5 Use the new label in your confusion matrix
```

```
conf_mat_res <- conf_mat(
  data = bank_clean,
  truth = status_label,
  estimate = kmeans_label
)
```

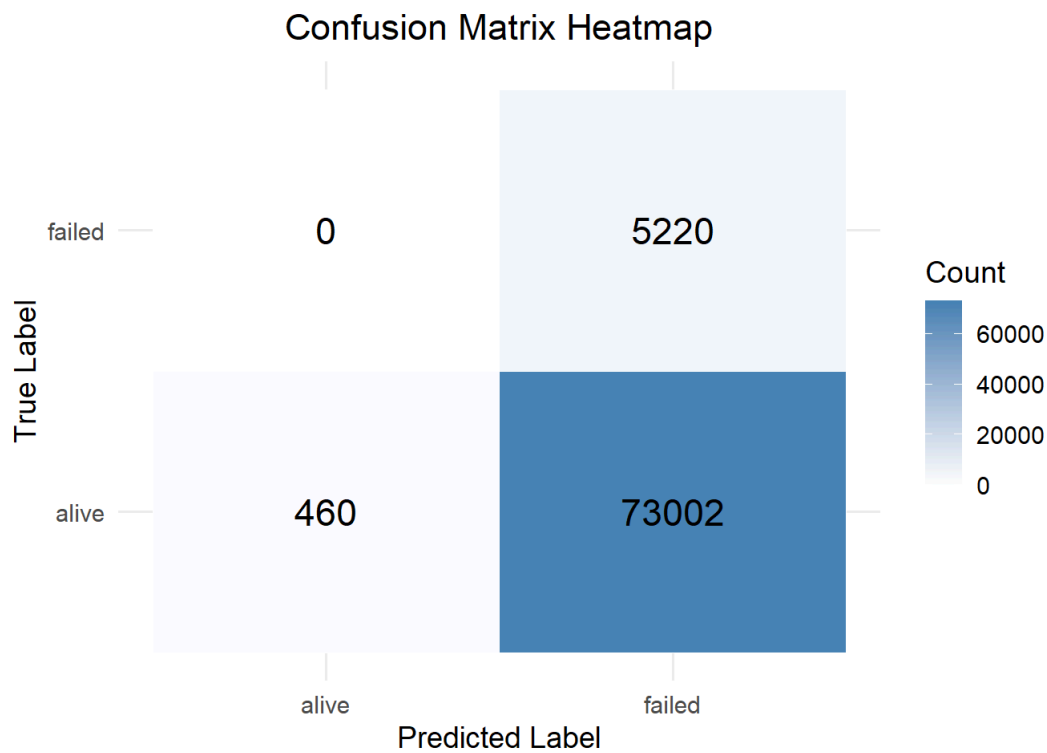
```
# 18.6 === Create confusion matrix ===
```

```
conf_mat_res <- conf_mat(
  data = bank_clean,
  truth = status_label,
  estimate = kmeans_label
)
```

```
# 18.7 === Confusion Matrix Heatmap ===
```

```
cm_df <- as.data.frame(conf_mat_res$table)

ggplot(cm_df, aes(x = Prediction, y = Truth, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), size = 6, color = "black") +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(
    title = "Confusion Matrix Heatmap",
    x = "Predicted Label",
    y = "True Label",
    fill = "Count"
  ) +
  theme_minimal(base_size = 14) +
  theme(plot.title = element_text(hjust = 0.5))
```



18.8 Train the model

```
model <- train(
  status_label ~ .,
  data = train_data,
  method = "rf",
  trControl = trainControl(method = "none"),
  tuneGrid = data.frame(mtry = 5),
  ntree = 50
)
```

18.9 Show all column names side-by-side
names(train_data)

```
## [1] "status_label"      "year"
## [3] "company_id"        "fiscal_year"
## [5] "total_assets"      "total_liabilities"
## [7] "current_assets"    "current_liabilities"
## [9] "working_capital"   "retained_earnings"
## [11] "sales"             "ebit"
## [13] "net_income"        "cash_flow_operations"
## [15] "cash_flow_investing" "cash_flow_financing"
## [17] "working_capital_to_assets" "retained_earnings_to_assets"
## [19] "ebit_to_assets"    "bankrupt_flag"
## [21] "kmeans_cluster"    "kmeans_label"
## [23] "years_from_2000"   "cluster"
```

names(test_data)

```
## [1] "status_label"      "year"
## [3] "company_id"        "fiscal_year"
## [5] "total_assets"      "total_liabilities"
## [7] "current_assets"    "current_liabilities"
## [9] "working_capital"   "retained_earnings"
## [11] "sales"             "ebit"
## [13] "net_income"        "cash_flow_operations"
## [15] "cash_flow_investing" "cash_flow_financing"
## [17] "working_capital_to_assets" "retained_earnings_to_assets"
## [19] "ebit_to_assets"    "bankrupt_flag"
## [21] "kmeans_cluster"    "kmeans_label"
## [23] "years_from_2000"
```

```
# 18.10 Show columns that are in train but NOT in test
setdiff(names(train_data), names(test_data))
```

```
## [1] "cluster"
```

18.11 Show columns that are in test but NOT in train

```
# 18.11 Show columns that are in test but NOT in train
setdiff(names(test_data), names(train_data))
```

```
## character(0)
```

```
# 18.12 Show columns that match exactly
intersect(names(train_data), names(test_data))
```

```
## [1] "status_label"      "year"
## [3] "company_id"        "fiscal_year"
## [5] "total_assets"      "total_liabilities"
## [7] "current_assets"    "current_liabilities"
## [9] "working_capital"   "retained_earnings"
## [11] "sales"             "ebit"
## [13] "net_income"        "cash_flow_operations"
## [15] "cash_flow_investing" "cash_flow_financing"
## [17] "working_capital_to_assets" "retained_earnings_to_assets"
## [19] "ebit_to_assets"    "bankrupt_flag"
## [21] "kmeans_cluster"    "kmeans_label"
## [23] "years_from_2000"
```

18.13 Remove the column

```
# 18.13 Remove the column
train_data <- train_data %>% select(-cluster)
```

```
# 18.14 Verify it's gone
# If it prints "FALSE" it good
"cluster_var" %in% names(train_data)
```

```
## [1] FALSE
```

```
# 18.15 Train the model
```

```
model <- train(
  status_label ~ .,
  data = train_data,
  method = "rf",
  trControl = trainControl(method = "none"),
  tuneGrid = data.frame(mtry = 5),
  ntree = 50
)
```

```
# 18.16 === Create and attach predictions BEFORE confusion matrix ===
```

```
pred_status <- predict(model, newdata = test_data)
```

```
test_data <- test_data %>%
  mutate(pred_status = pred_status)
```

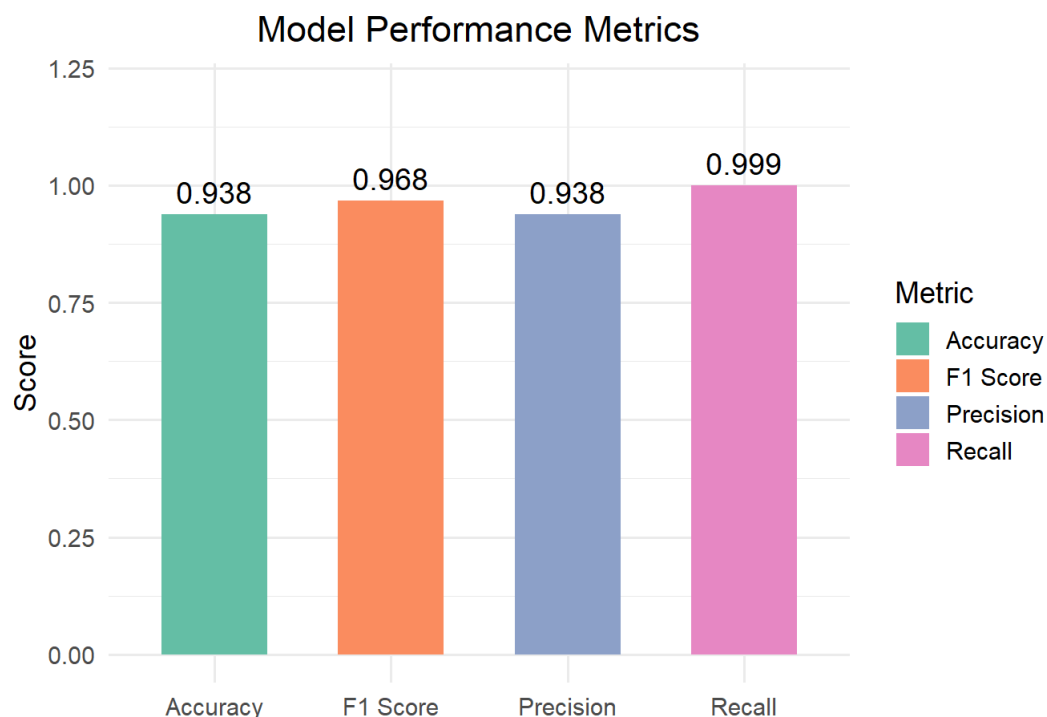
```
# 18.17 === Compute Metrics ===
```

```
accuracy_res <- accuracy(test_data, truth = status_label, estimate = pred_status)
precision_res <- precision(test_data, truth = status_label, estimate = pred_status)
recall_res <- recall(test_data, truth = status_label, estimate = pred_status)
f1_res <- f_meas(test_data, truth = status_label, estimate = pred_status)
```

```
# 18.18 === Metrics Bar Chart ===
```

```
metrics_df <- tibble(
  Metric = c("Accuracy", "Precision", "Recall", "F1 Score"),
  Value = c(
    accuracy_res$estimate,
    precision_res$estimate,
    recall_res$estimate,
    f1_res$estimate
  )
)
```

```
ggplot(metrics_df, aes(x = Metric, y = Value, fill = Metric)) +
  geom_col(width = 0.6) +
  geom_text(aes(label = round(Value, 3)), vjust = -0.5, size = 5) +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title = "Model Performance Metrics",
    x = "",
    y = "Score"
  ) +
  ylim(0, 1.2) +
  theme_minimal(base_size = 14) +
  theme(plot.title = element_text(hjust = 0.5))
```



STEP 19: Create a table of descriptive statistics

```
# 19.1 Descriptive table by status_label
descriptive_table <- bank_fe %>%
  group_by(status_label) %>%
  summarise(
    n = n(),
    mean_year = mean(year, na.rm = TRUE),
    sd_year = sd(year, na.rm = TRUE),
    mean_years_from_2000 = mean(years_from_2000, na.rm = TRUE),
    sd_years_from_2000 = sd(years_from_2000, na.rm = TRUE)
  )

descriptive_table
```

```
## # A tibble: 2 × 6
##   status_label      n mean_year sd_year mean_years_from_2000 sd_years_from_2000
##   <fct>         <int>   <dbl>  <dbl>             <dbl>         <dbl>
## 1 alive          73462    2008.   5.78              7.63          5.78
## 2 failed         5220     2006.   4.81              5.71          4.81
```

STEP 20: Save key outputs for R Markdown / report

```
# 20.1 You can save tables and metrics as objects to print nicely in R Markdown
cm_tbl <- tidy(conf_mat_res)
acc_val <- accuracy_res$.estimate
prec_val <- precision_res$.estimate
rec_val <- recall_res$.estimate
f1_val <- f1_res$.estimate

cm_tbl
```

```
## # A tibble: 4 × 2
##   name      value
##   <chr>    <dbl>
## 1 cell_1_1    460
## 2 cell_2_1 73002
## 3 cell_1_2     0
## 4 cell_2_2  5220
```

acc_val

```
## [1] 0.938019
```

prec_val

```
## [1] 0.9384563
```

rec_val

```
## [1] 0.9991379
```

f1_val

```
## [1] 0.9678469
```